

Local Search

Kaiqi Zhao

Harbin Institute of Technology, Shenzhen



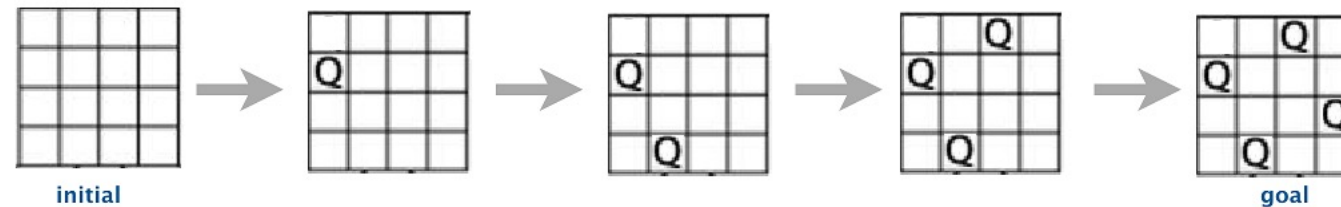
Systematic v.s. Local Search

- **Systematic search strategies:**
 - Solution is a sequence of actions
 - Maintain paths from start state to goal states
 - May be inefficient on large or possibly infinite search space.
- **Local search strategies:**
 - Do not keep track of the paths
 - Handle large or infinite state spaces where systematic search often fails to do

Constructive v.s. Perturbative Search

- **Constructive Search:**

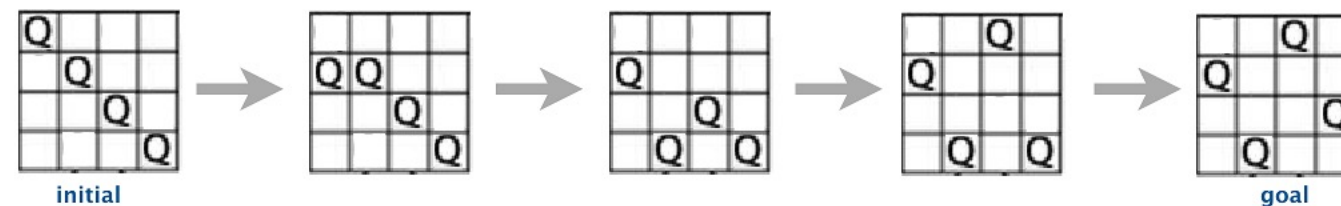
- Start with an empty solution and generate candidate solutions by iteratively extending partial solutions



- Search space: All partial and full solutions

- **Perturbative Search :**

- Start with a candidate solution and find the desired solution by iteratively modifying one or more features of the present solution.



- Search space: All full solutions

Feature-based Representation

- A **feature-based representation** involves features $X_1, \dots, X_k$.
- Candidate solutions are assignments that map each feature X_i in the set with some $x_i \in Dom(X_i)$, denoted by $(x_1, \dots, x_k)$
 - $Dom(X_i)$ defines the domain of the feature X_i , which includes all possible values of X_i
- **Local search** strategies are often based on perturbative search, using feature-based representations.

Local Search

Given a search problem, a local search strategy for solving an arbitrary instance of this problem is defined by:

- A **search space** S : the set of candidate solutions over features $X_1 \dots, X_k$.
- **Initialisation** of a candidate solution.
- **Successor relation** on S .
- **Step function** to define how a solution is transformed into another
- **Termination function** that specifies when to stop the search

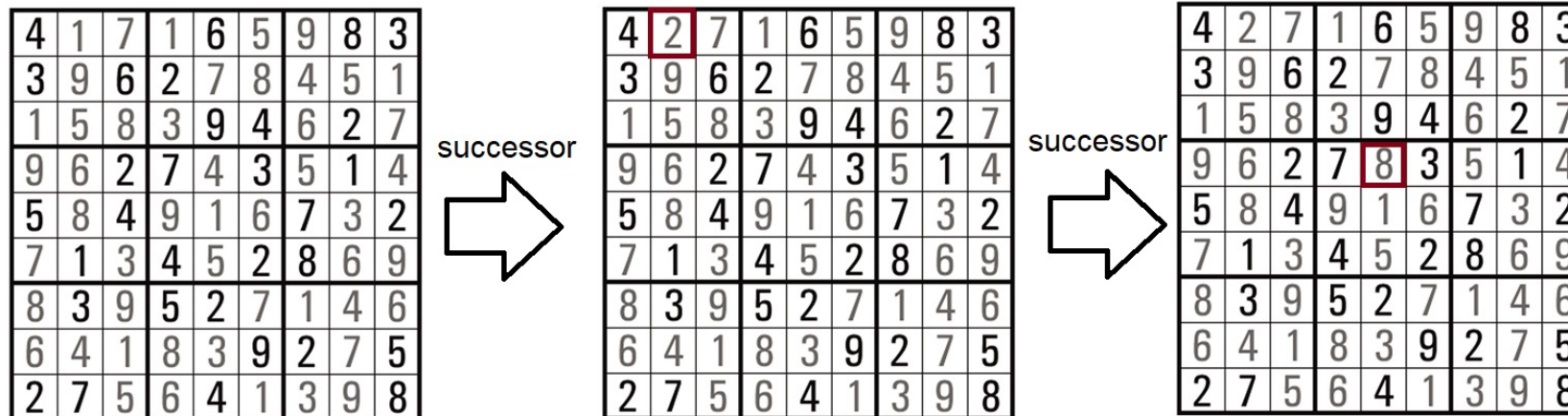
Local Search

Local search strategies are heuristics to solve problems with large search space.

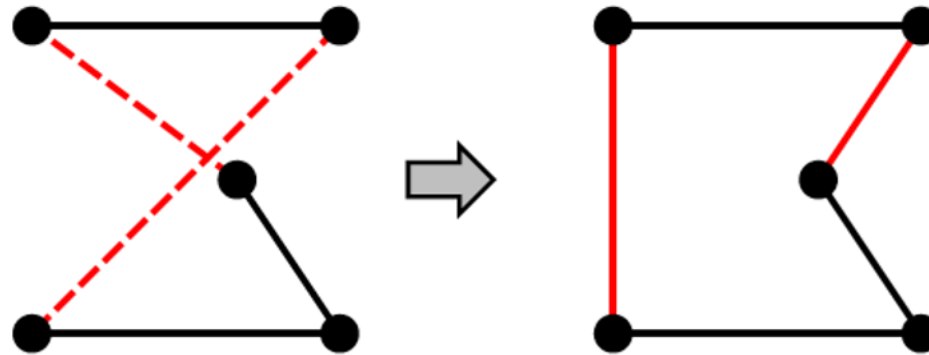
- Given a candidate solution $\{X_1 = x_1, \dots, X_k = x_k\}$, a **successor** is a candidate solution $\{X_1 = y_1, \dots, X_k = y_k\}$ where $x_i = y_i$ for all but one $i \in \{1, \dots, k\}$.
- A local search traverses the space of candidate solutions, generating a sequence of candidate solutions

$$\omega_0, \omega_1, \omega_2, \dots$$

so that ω_{j+1} is a successor of ω_j for all $j \geq 0$.



Example. [Traveling salesman problem] A **tour** through n given cities is a path that visits every city once and exactly once, and returns to the starting city.
TSP asks for a **tour** with the shortest total distance traversed.



A successor relation is 2-exchange relation

The Generic LocalSearch Procedure

INPUT: A search problem with features $X_1, \dots, X_k$, domain dom

OUTPUT: A solution $(x_1, \dots, x_k)$

for $i = 1, 2, \dots, k$ **do**

$x_i \leftarrow$ a random value in $dom(X_i)$

end for

while **stopping criterion** is not met **do**

Select a successor $(y_1, \dots, y_k)$

Set $(x_1, \dots, x_k) \leftarrow (y_1, \dots, y_k)$

end while

return $(x_1, \dots, x_k)$

▷ A **try**

▷ Random Initialisation

▷ A **local search**

▷ Select a successor

A **local search strategy** can be defined by specifying how features are selected and improved at each iteration, and what stopping criterion is applied.

Two Trivial Local Search Strategies

- **Random sampling:** The stopping criterion is always true (so no local search is performed). Each try involves taking a random candidate solution.
- **Random walk:** Each local search iteration involves changing the value of a feature at random.

Pros and cons with these methods:

- **Advantages:** The optimal solution can be generated at each try with positive probability. So it will eventually be found.
- **Disadvantages:** There is no upper bound on the time it takes.

Basic Local Search: Iterative Best Improvement (IBI)

A local search strategy defines how to select the successor of a candidate solution.

Iterative Best Improvement

Iterative best improvement is a local search strategy that always selects a successor that minimises loss (or maximises gain), with random tie breaks, i.e., it performs **greedy choice**.

$$A' := \operatorname{argmin}_{\text{Successor } A'} \text{val}(A') \text{ or } A' := \operatorname{argmax}_{\text{Successor } A'} \text{val}(A')$$

- $\text{val}(\cdot)$ is an evaluation function that measures how good a solution is.

- For minimisation, the method is called **greedy descent**;
- For maximisation, the method is called hill climbing.¹



¹We only discuss greedy descent.

GreedyDescent

INPUT: A search problem with features $X_1, \dots, X_k$, domain dom , evaluation function val

OUTPUT: A solution $(x_1, \dots, x_k)$

for $i = 1, 2, \dots, k$ **do**

$x_i \leftarrow$ a random value in $dom(X_i)$

▷ Random Initialisation

end for

while true **do**

Find a successor $(x_1, \dots, x_{i-1}, y, x_{i+1}, \dots, x_k)$ that minimises the val function.

if the found successor has value no less than $(x_1, \dots, x_k)$ **then**

return $(x_1, \dots, x_k)$

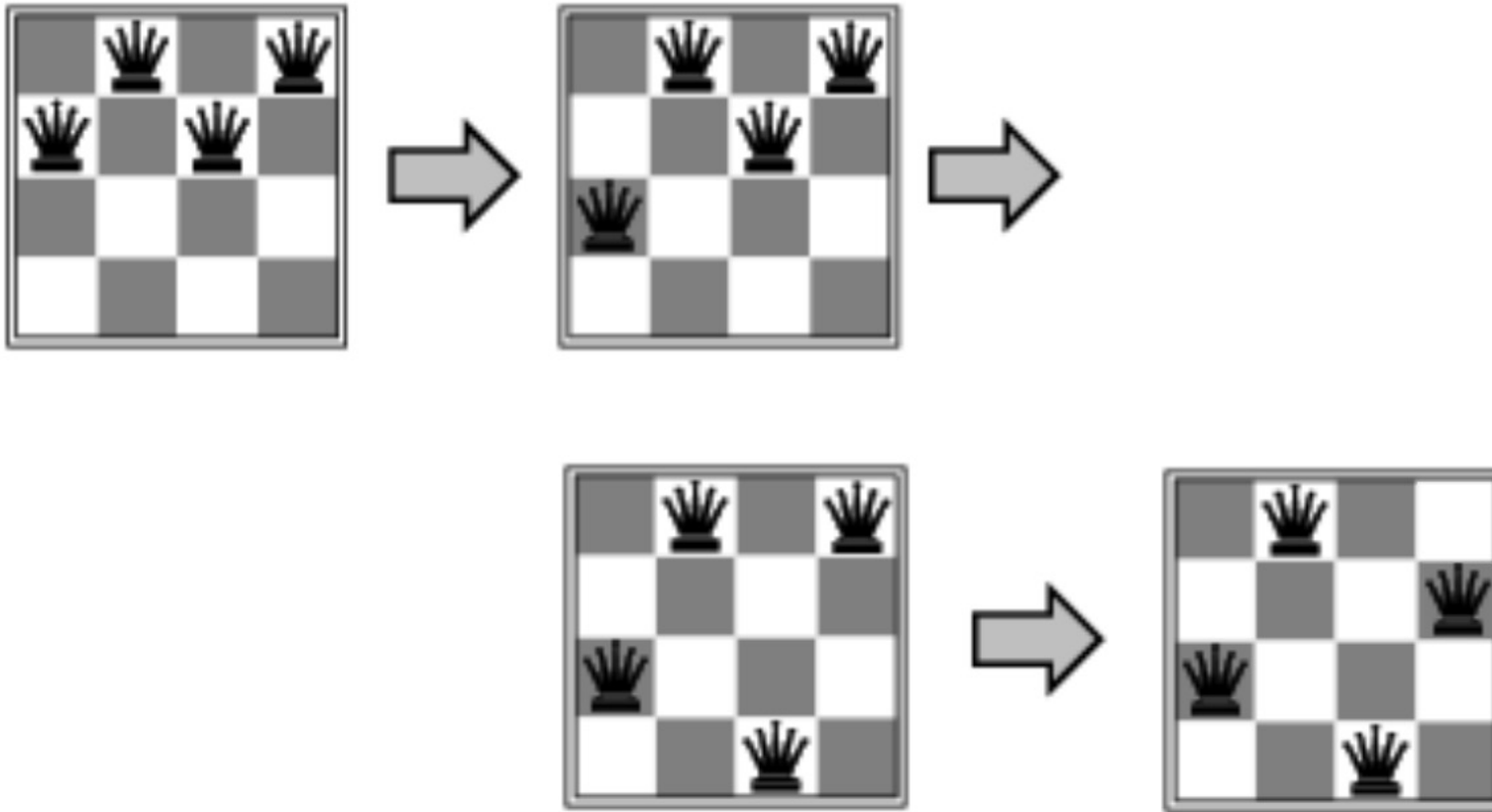
▷ Termination

end if

$x_i \leftarrow y$

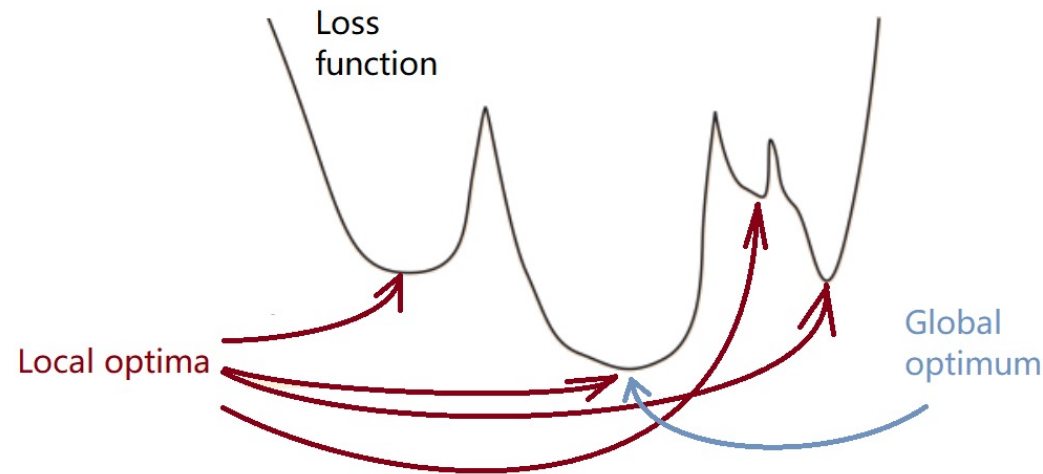
end while

Example. [Four queens] Consider the four-queen problem with the initialisation below. Two solutions are **successors** of each other if they differ by at most **one feature**.



Limitations of IBI

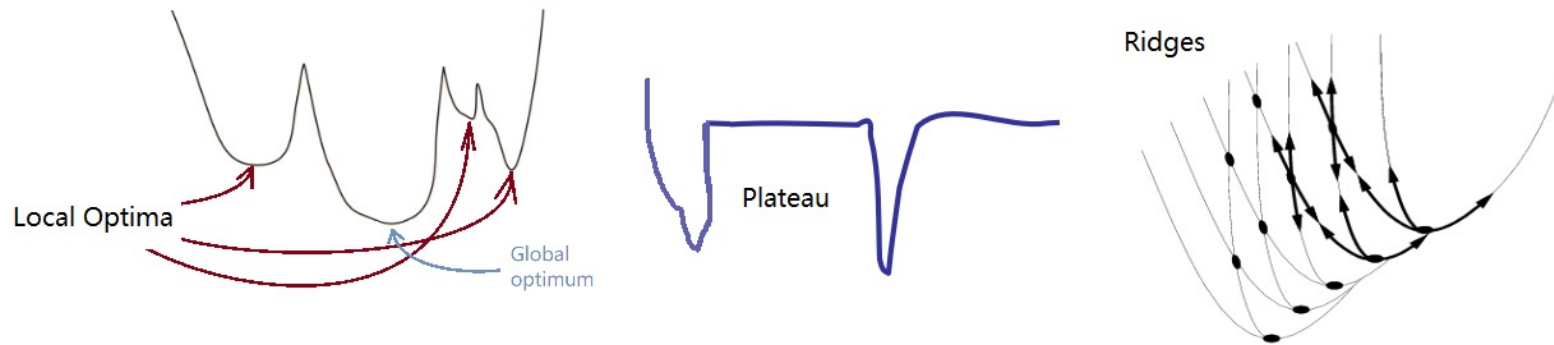
- A **local optimum** is a candidate solution such that no possible successor improves the evaluation function.
- A **global optimum** is a candidate solution that has the best value for the evaluation function out of all possible candidate solutions.



A heuristic is **complete** if it guarantees to find a (globally) optimal solution. Iterative best improvement is not complete as it can get stuck in local optima.

Limitations of IBI

- A **plateau** is a region of the search space of where no local move decreases cost.
- A **ridge** is a sequence of local optima that are not directly connected to each other.



IBI will stop moving on plateau. When it gets trapped in ridges, it is hard for it to get to better local optimum.

We are going to introduce a number of local search strategies:

- Random restarts
- Sideway moves
- Iterative first improvements (IFI)
- Large neighbourhood

Variant 1: Random Restarts

Greedy descent with random restarts runs a sequence of greedy descent searches:

1. Run greedy descent from a random initial candidate
2. Repeat step 1.

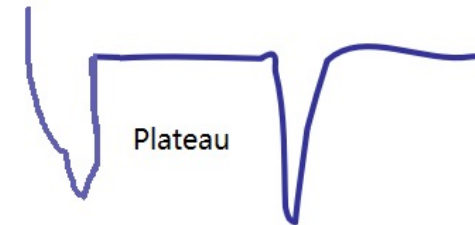
Different variants of this strategy:

- For each restart, run until termination or run for a fixed number of steps
 - Run a fixed number of restarts or run indefinitely.
-
- Whether a try is successful or not is a **Bernoulli random variable**
 - If a success rate of a single try is p , then the expected number of tries needed to find the optimal solution is $1/p$

Variant 2: Sideway Moves

Greedy descent with sideway moves runs greedy descent searches with some adjustments to handle plateau:

- If no improvement can be found, move **sideways** (successors with the same value)
- Set a parameter m such that only $\leq m$ consecutive sideway moves can be performed



Variant 3: Iterative First Improvement

Iterative First Improvement chooses the next candidate as follows:

- Iterate over the successors (in certain order) of the current candidate x and find the **first improving candidate** y :

$$val(y) < val(x)$$

- Set the current solution to y

Advantage: Faster step-wise time complexity when many successors exist.

Disadvantage: Performance depends on the ordering of the successors.

Variant 4: Large Neighbourhoods

The **neighbourhood** of a candidate solution $x = (x_1, \dots, x_k)$ is the set of all candidate solutions that are within 1-step of the successor relation $\rightsquigarrow$, denoted by:

$$N(x) = \{(y_1, \dots, y_k) \mid (x_1, \dots, x_k) \rightsquigarrow (y_1, \dots, y_k)\}$$

Define the successor relation $\rightsquigarrow$ on the two solutions if they are different in at most ρ features, where $1 \leq \rho \leq k$.

- Greedy Descent: $\rho = 1$
- Systematic Search: $\rho = k$ (**all neighbours**)

Variant 4: Large Neighbourhoods

Advantage: Increase the chance of finding good solutions in fewer steps

Disadvantage: dramatically increase time-complexity for each step.

The neighbourhood grows in **exponential**: $O((kd)^\rho)$, where d is the maximum number of possible values of a feature $d = \max\{|dom(X_i)|, 1 \leq i \leq k\}$

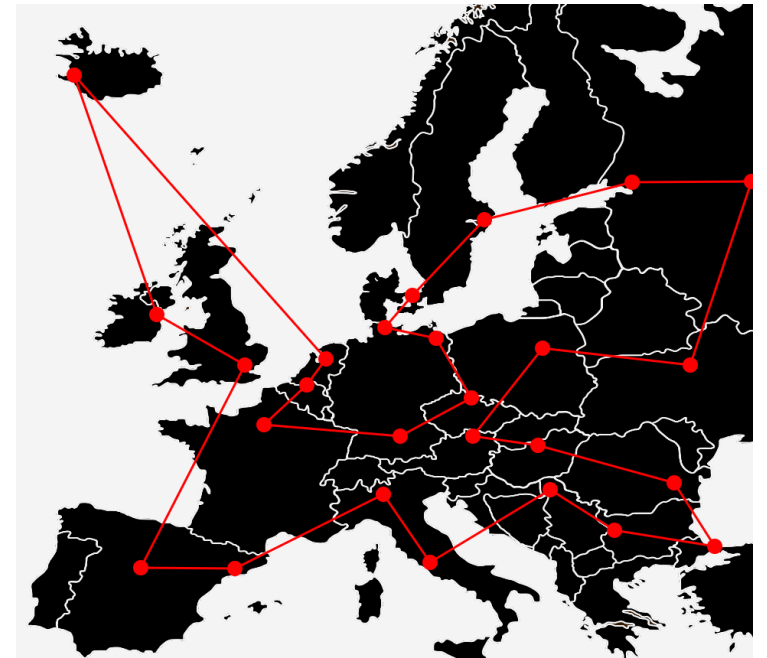
Techniques to reduce the computation for large neighbourhoods:

1. Neighbourhood Pruning
2. Variable Neighbourhood Descent (VND)

Technique 1: Neighbourhood Pruning

- Prune successors that are unlikely to yield improvement
- Maintain a candidate list of possible successors for each candidate solution
- Only move to successors in the candidate list

Example. In TSP, the candidate list can contain only cities that are within a certain distance threshold.



Technique 2: Variable Neighbourhood Descent (VND)

- Use k successor relations, ordered according to increasing size of the respective neighbourhoods $N_1, N_2, \dots, N_k$.
- Start with neighbourhood N_1 and perform greedy descent
- When no improvement can be found for neighbourhood N_i , search in neighbourhood N_{i+1} .
- If an improvement is found, go back to N_1

Advantage: VND can considerably improve the performance of greedy descent both with respect to solution quality and search time.

Exercise. Suppose there are five feature A, B, C, D, E with constraints

$$B \neq 3, C \neq 2, A \neq B, B \neq C, C < D, A = D, \\ E < A, E < B, E < C, E < D, B \neq D$$

Each feature can take values from $\{1, 2, 3, 4\}$

Define the evaluation function as

$$\text{val}(\{A = a, \dots, E = e\}) = \text{the number of violated constraints}$$

Start from an initial solution $A=B=C=D=E=2$. **Apply greedy descent**, write down the sequence of solutions obtained

Stochastic Local Search

Recall: Local Search

Given a search problem, a local search strategy for solving an arbitrary instance of this problem is defined by:

- A **search space** S : the set of candidate solutions over features $X_1 \dots, X_k$.
- **Initialisation** of a candidate solution.
- **Successor relation** on S .
- **Step function** to define how a solution is transformed into another
- **Termination function** that specifies when to stop the search

Recall: Local Search

Local search strategy performs the following steps:

- Starting from an initial candidate solution $(x_1, x_2, \dots, x_k)$
- Iteratively move to successor solutions
- Until certain stopping criterion is met

A local search traverses the space of candidate solutions, generating a sequence of candidate solutions

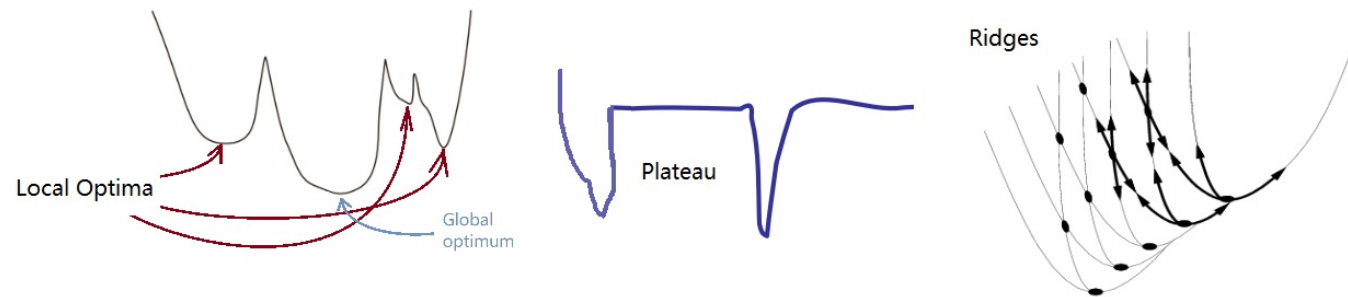
$$\omega_0, \omega_1, \omega_2, \dots$$

so that ω_{j+1} is a successor of ω_j for all $j \geq 0$.

Recall: Local Search

Problems with Iterative Best Improvement (greedy descent, hill climbing):

- Local optimum
- Plateau
- Ridge



We have discussed several strategies to improve greedy descent

- Greedy descent does not guarantee to find the optimal solution
- Greedy descent **with random restarts** will find the optimal solution, given sufficiently many restarts are performed.

Stochastic Local Search (SLS)

Stochastic local search (SLS) are local search strategies with **randomisation**

- **Initialisation:** Random start
- **Tie breaking:** Random tie breaking
- **Step function:** e.g. random ordering of successors (iterative first improvement)

The local search strategies that we introduced earlier can all be implemented as stochastic local search.

We will focus on more SLS strategies that **randomness** is implanted into the **step function**.

Key Idea: The search process can perform worsening steps which help it to escape from local optima.

SLS 1: Randomised Iterative Improvement (RII)

RII assumes that adding randomisation may avoid the search stuck in local optima.

Main idea:

- Pick a parameter (walk probability) $p \in [0, 1]$
- At every step,
 - With probability p , make a **uninformed random walk**, i.e., choose a successor **uniformly at random** and move there.
 - With probability $1 - p$, make a greedy choice: choose the successor that has the least loss.

What is the expected length of a consecutive sequence of random walk steps?

Formal analysis of RII

Probabilistic Approximately Complete

A search strategy is **probabilistic approximately complete** (PAC) if the probability that a run fails to find an optimal solution can be made arbitrarily small when the search runs for sufficiently long.

Uniform Improvement Property

A local search strategy has the **uniform improvement property** if there exists $\varepsilon > 0$ such that in each step, the distance to an arbitrary, but fixed optimal solution s^* is reduced with **probability $\geq \varepsilon$** .

Lemma

Suppose the search space S is finite. If a local search strategy has uniform improvement property, then it is PAC.

Lemma

Suppose the search space S is finite. If a local search strategy has **uniform improvement property**, then it is **PAC**.

Proof.

- Suppose the current solution s has distance l to the optimal solution s^* .
- By **uniform improvement property**, the probability that a sequence of steps moving the current candidate from s to s^* is $\geq \varepsilon^l$.
- Denote the the diameter of the search space as Δ .
- Therefore, from any candidate, the probability that a try fails is at most
$$(1 - \varepsilon^\Delta)^{\lfloor t/\Delta \rfloor}$$
- Since $\varepsilon^\Delta > 0$, by choosing t sufficiently large, this probability can be made arbitrarily small.

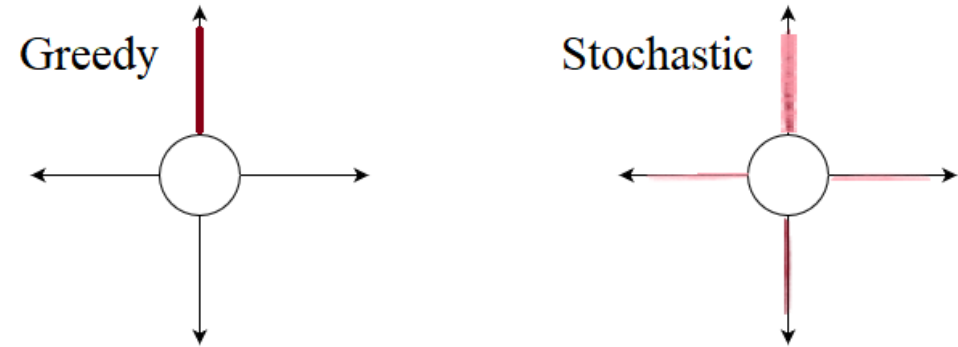
Fact. RII has the **uniform improvement property** with $\varepsilon = p/b$, where b is the branching factor.

Thus, RII is **PAC**.

SLS 2: Probabilistic Iterative Improvement (PII)

Probabilistic iterative improvement (PII)

allows worsening search steps with a probability that depends on the respective **deterioration** in evaluation function value.



Main idea:

- Choosing the successor that decreases loss the most may lead to local optima that are not global optima.
- To go out of local optima, it is necessary to choose other successors.
- We are still favouring successors that bring faster decrease on the loss.
- We will turn “hard max” to “soft max”.

Softmax Function

A **softmax function** σ , also known as **normalised exponential**, normalises a vector of K real numbers into a probability distribution consisting of K probabilities: For $\vec{z} = (z_1, \dots, z_K) \in \mathbb{R}^K$,

$$\sigma(\vec{z})_i = \frac{e^{\beta z_i}}{\sum_{j=1}^K e^{\beta z_j}} \text{ where } i = 1, \dots, K$$

where $\beta \in \mathbb{R}$ is a parameter.

Note. Softmax is a “smooth” maximum function.

Example. Set $\beta = 1$. $\sigma(0,10) \approx (0.00005, 0.99995)$.
Set $\beta = 0.01$. $\sigma(0,10) \approx (0.475, 0.525)$.

PII selects a successor by computing the probability distribution $p_x(y)$ using the softmax function, i.e.,

$$p_x(y) = \begin{cases} 1 & \text{if } val(y) < val(x) \\ \frac{e^{-\beta(val(y)-val(x))}}{\sum_{z \in N(x)} e^{-\beta(val(z)-val(x))}} & \text{otherwise} \end{cases}$$

Note. The softmax function ensures that the successors with a better evaluation are more likely to be chosen.

LSL 3: Simulated Annealing

Annealing is a metal-lurgical process where melted metals are slowly cooled to allow them to reach a low energy state, making them stronger.

- At high temperature, the thermodynamic movement tends to be more random.
- At low temperature, there is little random movements.



The annealing process would start by keeping the metal above its recrystallisation temperature for a while before letting it cool down.

Boltzmann Distribution

Suppose a system (consisting of particles) can be in one of M states in S where each state has different energy and temperature.

Boltzmann distribution is a probability distribution for the system to be in a certain state as a function of that state's energy and temperature, i.e., the probability for state i is computed by a softmax function

$$p_i = \frac{e^{-\frac{\varepsilon_i}{Kt}}}{\sum_{j=1}^M e^{-\frac{\varepsilon_j}{Kt}}} \propto e^{-\frac{\varepsilon_i}{Kt}}$$

where ε_i is the energy of state i , K is Boltzmann's constant, and t is the thermodynamic temperature.

- The states with lower energy will always have a higher probability of being occupied than the states with higher energy.
- The **Boltzmann factor** is the ratio between the probabilities of two states

$$\frac{p_i}{p_j} = e^{-\frac{\varepsilon_i - \varepsilon_j}{Kt}}$$

Simulated Annealing

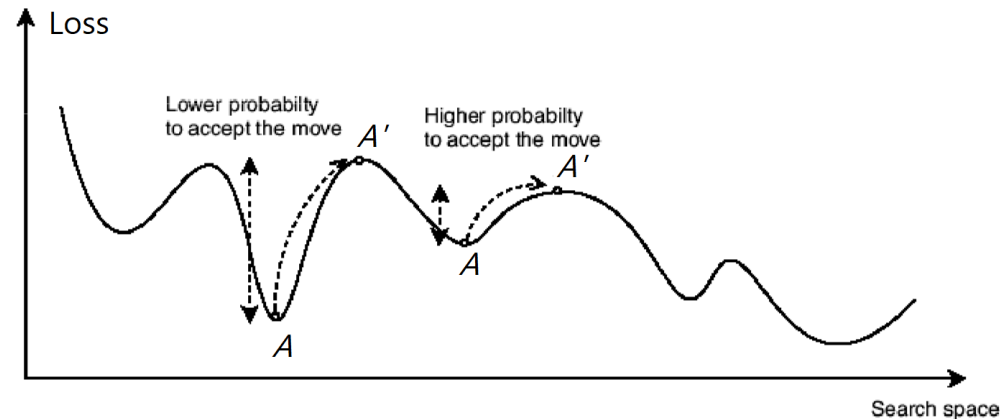
Simulated annealing is a local search strategy that simulates the annealing process in real life.

- It maintains a **temperature** value $T > 0$ to control the optimisation steps.
- It considers $val(A)$ of a total assignment $\{X_1 = x_1, \dots, X_k = x_k\}$ as an **energy** value of the candidate A .
- We use the Boltzmann factor as the probabilities of adopting a successor A' from candidate solution A . Let $\Delta\varepsilon = val(A') - val(A)$. The probability is

$$e^{-\frac{\Delta\varepsilon}{T}}$$

Suppose $\text{val}(A') > \text{val}(A)$. Then $\Delta\varepsilon > 0$

- If temperature T is high, then $e^{\frac{-\Delta\varepsilon}{T}}$ is close to 1.
The successor will be very likely to be chosen.
- If T is low, then $e^{\frac{-\Delta\varepsilon}{T}}$ is close to 0.
The successor will be very unlikely to be chosen.
- At an intermediate temperature, the algorithm will tend to accept the successor with a lower energy difference, but may also accept a successor with a higher energy difference.



Simulated Annealing

At every iteration, when selecting a successor of the current candidate solution A :

- Pick a feature X at random, and then pick a value $x \in \text{dom}(X)$ at random.
- Let A' be the resulting successor by setting $X = x$.
- If $\text{val}(A') \leq \text{val}(A)$, choose A' as the successor, and proceed to the next step.
- If $\text{val}(A') > \text{val}(A)$, choose A' as the successor using a Boltzmann distribution, i.e., with probability

$$e^{-(\text{val}(A') - \text{val}(A))/T}$$

Simulated Annealing Local Search

INPUT: Candidate solution A

OUTPUT: Successor A'

Generate a random successor A'

if $\text{val}(A') - \text{val}(A) < 0$ **do**

return A'

else

return A' with probability

$$e^{-(\text{val}(A') - \text{val}(A))/T}$$

$T \leftarrow \text{UpdateTemp}(T)$

end if

▷ annealing schedule

- An **annealing schedule** is a procedure that reduces the temperature.
- One common annealing schedule is **geometric cooling**, e.g.,
 - Start with $T \leftarrow 10$.
 - After every iteration, let $T \leftarrow T \times 0.99$.

A good annealing schedule requires some **tuning process**.

Example. In the table below, A' is called ***i*-worse** if $\text{val}(A') - \text{val}(A) = i$.

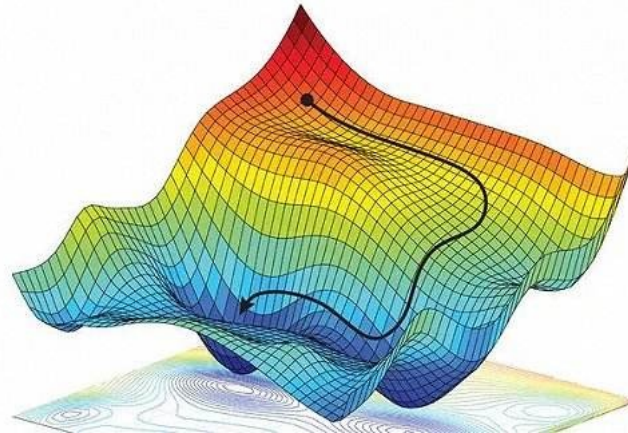
Temperature	Probability of acceptance		
	1-worse	2-worse	3-worse
10	0.9	0.82	0.74
1	0.37	0.14	0.05
0.25	0.018	0.0003	0.000006
0.1	0.00005	$2 * 10^{-9}$	$9 * 10^{-14}$

Local Search 4: Gradient Descent

In many optimization problem:

- Some feature has continuous domain, e.g., $\text{dom}(X) = \mathbb{R}$.
- The loss function val is defined as a continuous and differentiable function on the features.

In this case, **gradient descent** aims to minimise the loss function by simulating how water flows downhill: When choosing a direction to go, always pick the direction that goes down the most.

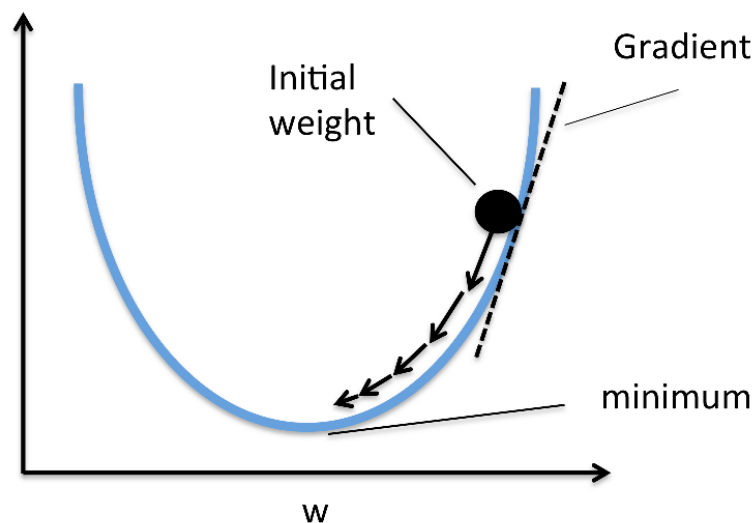


If there is only one feature X :

- Suppose x is the current value of X .
- The next value is

$$x' := x - \eta \times \frac{d\text{val}(x)}{dX}$$

where η , the **step size**, is the constant of proportionality that determines how fast gradient descent approaches the minimum, and $d\text{val}(v)/dX$ is the derivative of val with respect to X .



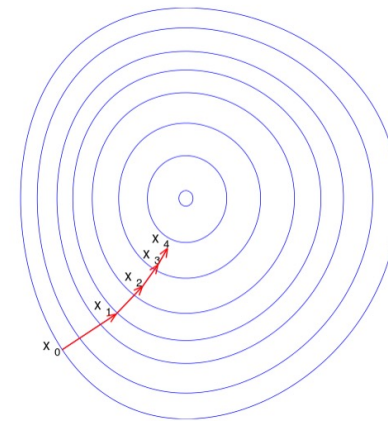
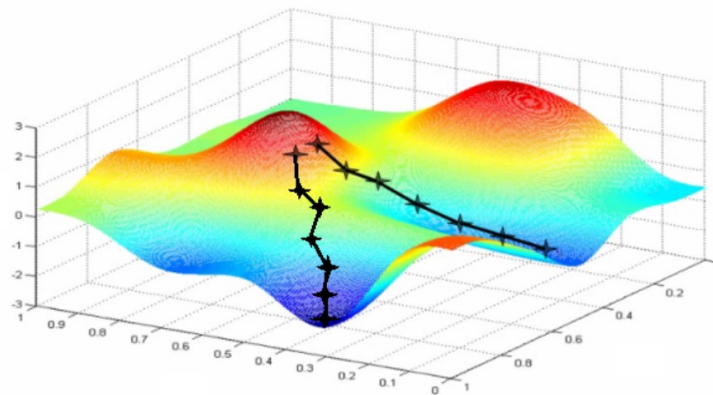
Note. If η is too large, gradient descent may overshoot the minimum; if η is too small, progress becomes very slow.

For the general case when we have features $X_1, \dots, X_k$:

- Suppose the current candidate solution is $\omega = \{X_1 = x_1, \dots, X_k = x_k\}$.
- In the successor, after gradient descent applies to X_i , has X_i taking the value

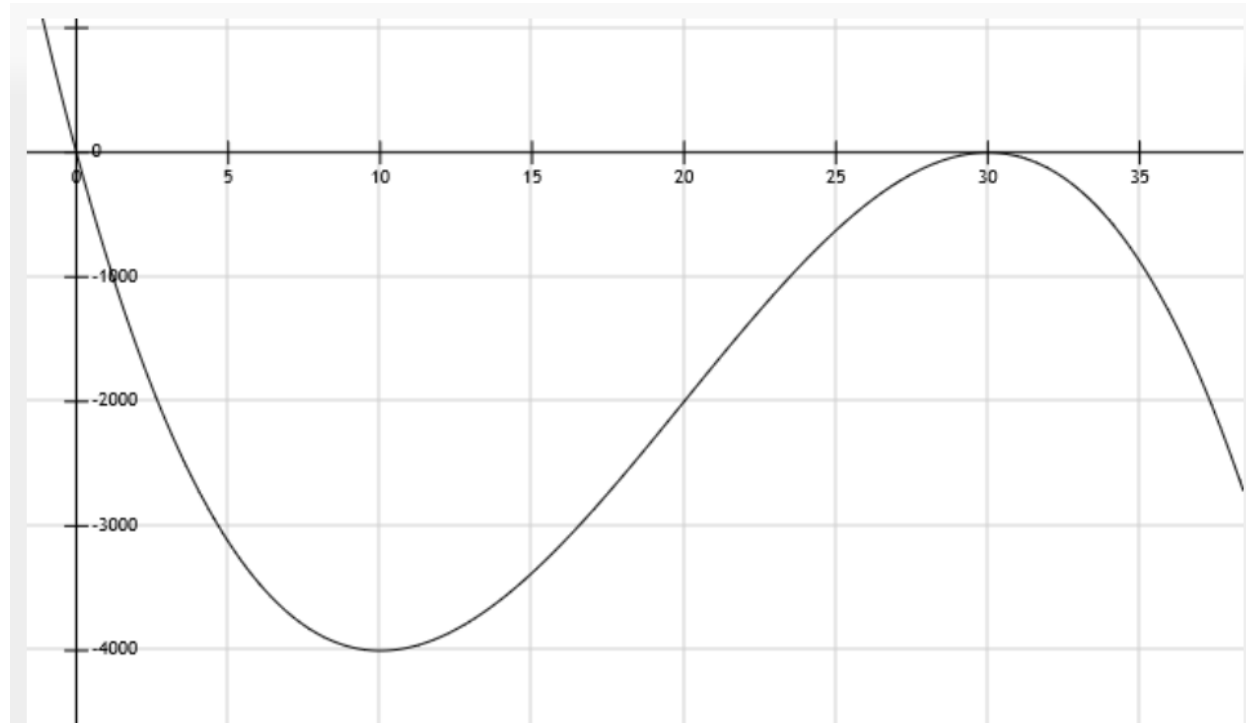
$$x_i' := x_i - \eta \times \frac{\partial \text{val}(x_1, \dots, x_k)}{\partial X_i}$$

In practice, gradient descent normally goes through and applies to every feature in a single iteration.



Example. Consider the function $f(x) = -x^3 + 60x^2 - 900x - 1$. Now imagine that the domain of x is the set of real numbers $\mathbb{R}$.

Perform 4 iterations of gradient descent algorithm starting from $x = 0$, with the following settings:



Summary of The Topic

The following are the main knowledge points covered:

- What is local search?
- What is the difference between local and global optima?
- What is the general framework of a local search strategy?
- Describe the Iterative Best Improvement method.
- Describe the variants of IBI.
- Describe the stochastic local search strategies.